CAR DEKHO MODEL PRICE PREDECTION

In [1]:
```
import pandas as pd
df = pd.read_csv("cardekho_dataset.csv")
df
```

Out[1]:

	Unnamed: 0	car_name	brand	model	vehicle_age	km_driven	seller_type	fuel_type	transmission_type	mileage	engin
0	0	Maruti Alto	Maruti	Alto	9	120000	Individual	Petrol	Manual	19.70	79
1	1	Hyundai Grand	Hyundai	Grand	5	20000	Individual	Petrol	Manual	18.90	119
2	2	Hyundai i20	Hyundai	i20	11	60000	Individual	Petrol	Manual	17.00	119
3	3	Maruti Alto	Maruti	Alto	9	37000	Individual	Petrol	Manual	20.92	99
4	4	Ford Ecosport	Ford	Ecosport	6	30000	Dealer	Diesel	Manual	22.77	149
...	...	...	...	...	...	...	...	...	...	...	...
15406	19537	Hyundai i10	Hyundai	i10	9	10723	Dealer	Petrol	Manual	19.81	108
15407	19540	Maruti Ertiga	Maruti	Ertiga	2	18000	Dealer	Petrol	Manual	17.50	137
15408	19541	Skoda Rapid	Skoda	Rapid	6	67000	Dealer	Diesel	Manual	21.14	149
15409	19542	Mahindra XUV500	Mahindra	XUV500	5	3800000	Dealer	Diesel	Manual	16.00	217
15410	19543	Honda City	Honda	City	2	13000	Dealer	Petrol	Automatic	18.00	149

15411 rows × 14 columns

step 1 -> Understanding the data

In [2]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15411 entries, 0 to 15410
Data columns (total 14 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   Unnamed: 0         15411 non-null  int64  
 1   car_name           15411 non-null  object 
 2   brand              15411 non-null  object 
 3   model              15411 non-null  object 
 4   vehicle_age        15411 non-null  int64  
 5   km_driven          15411 non-null  int64  
 6   seller_type        15411 non-null  object 
 7   fuel_type          15411 non-null  object 
 8   transmission_type  15411 non-null  object 
 9   mileage            15411 non-null  float64
 10  engine             15411 non-null  int64  
 11  max_power          15411 non-null  float64
 12  seats              15411 non-null  int64  
 13  selling_price      15411 non-null  int64  
dtypes: float64(2), int64(6), object(6)
memory usage: 1.6+ MB
```

In [3]:
```
df.shape
```

Out[3]: (15411, 14)

In [4]:
```
df.describe()
```

Out[4]:

	Unnamed: 0	vehicle_age	km_driven	mileage	engine	max_power	seats	selling_price
count	15411.000000	15411.000000	1.541100e+04	15411.000000	15411.000000	15411.000000	15411.000000	1.541100e+04
mean	9811.857699	6.036338	5.561648e+04	19.701151	1486.057751	100.588254	5.325482	7.749711e+05
std	5643.418542	3.013291	5.161855e+04	4.171265	521.106696	42.972979	0.807628	8.941284e+05
min	0.000000	0.000000	1.000000e+02	4.000000	793.000000	38.400000	0.000000	4.000000e+04
25%	4906.500000	4.000000	3.000000e+04	17.000000	1197.000000	74.000000	5.000000	3.850000e+05
50%	9872.000000	6.000000	5.000000e+04	19.670000	1248.000000	88.500000	5.000000	5.560000e+05
75%	14668.500000	8.000000	7.000000e+04	22.700000	1582.000000	117.300000	5.000000	8.250000e+05
max	19543.000000	29.000000	3.800000e+06	33.540000	6592.000000	626.000000	9.000000	3.950000e+07

In [5]:
```
df.describe(include="object")
```

Out[5]:

	car_name	brand	model	seller_type	fuel_type	transmission_type
count	15411	15411	15411	15411	15411	15411
unique	121	32	120	3	5	2
top	Hyundai i20	Maruti	i20	Dealer	Petrol	Manual
freq	906	4992	906	9539	7643	12225

In [6]:
```
df.columns
```

Out[6]:
```
Index(['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven',
       'seller_type', 'fuel_type', 'transmission_type', 'mileage', 'engine',
       'max_power', 'seats', 'selling_price'],
      dtype='object')
```

In [7]:
```
df.duplicated().sum()
```

Out[7]:
```
np.int64(0)
```

In [8]:
```
df.isnull().sum()
```

Out[8]:
```
Unnamed: 0           0
car_name             0
brand                0
model                0
vehicle_age          0
km_driven            0
seller_type          0
fuel_type            0
transmission_type    0
mileage              0
engine               0
max_power            0
seats                0
selling_price        0
dtype: int64
```

In [9]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

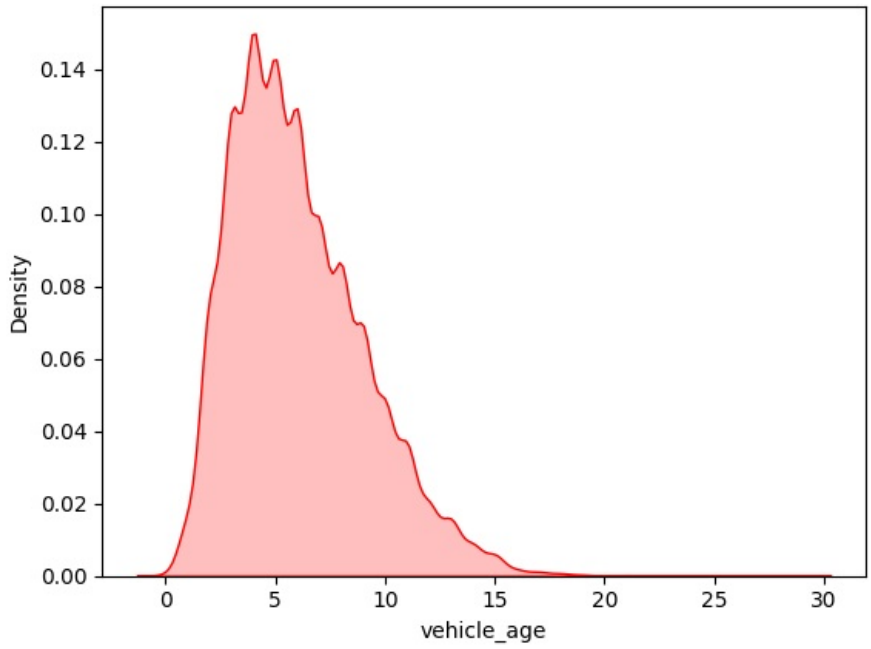
In [10]:
```
#spelling check
df["car_name"].unique()
```

Out[10]:
```
array(['Maruti Alto', 'Hyundai Grand', 'Hyundai i20', 'Ford Ecosport',
       'Maruti Wagon R', 'Hyundai i10', 'Hyundai Venue', 'Maruti Swift',
       'Hyundai Verna', 'Renault Duster', 'Mini Cooper', 'Maruti Ciaz',
       'Mercedes-Benz C-Class', 'Toyota Innova', 'Maruti Baleno',
       'Maruti Swift Dzire', 'Volkswagen Vento', 'Hyundai Creta',
       'Honda City', 'Mahindra Bolero', 'Toyota Fortuner', 'Renault KWID',
       'Honda Amaze', 'Hyundai Santro', 'Mahindra XUV500',
       'Mahindra KUV100', 'Maruti Ignis', 'Datsun RediGO',
       'Mahindra Scorpio', 'Mahindra Marazzo', 'Ford Aspire', 'Ford Figo',
       'Maruti Vitara', 'Tata Tiago', 'Volkswagen Polo', 'Kia Seltos',
       'Maruti Celerio', 'Datsun GO', 'BMW 5', 'Honda CR-V',
       'Ford Endeavour', 'Mahindra KUV', 'Honda Jazz', 'BMW 3', 'Audi A4',
       'Tata Tigor', 'Maruti Ertiga', 'Tata Safari', 'Mahindra Thar',
       'Tata Hexa', 'Land Rover Rover', 'Maruti Eeco', 'Audi A6',
       'Mercedes-Benz E-Class', 'Audi Q7', 'BMW Z4', 'BMW 6', 'Jaguar XF',
       'BMW X5', 'MG Hector', 'Honda Civic', 'Isuzu D-Max',
       'Porsche Cayenne', 'BMW X1', 'Skoda Rapid', 'Ford Freestyle',
       'Skoda Superb', 'Tata Nexon', 'Mahindra XUV300',
       'Maruti Dzire VXI', 'Volvo S90', 'Honda WR-V', 'Maruti XL6',
       'Renault Triber', 'Lexus ES', 'Jeep Wrangler', 'Toyota Camry',
       'Hyundai Elantra', 'Toyota Yaris', 'Mercedes-Benz GL-Class',
       'BMW 7', 'Maruti S-Presso', 'Maruti Dzire LXI', 'Hyundai Aura',
       'Volvo XC', 'Maserati Ghibli', 'Bentley Continental', 'Honda CR',
       'Nissan Kicks', 'Mercedes-Benz S-Class', 'Hyundai Tucson',
       'Tata Harrier', 'BMW X3', 'Skoda Octavia', 'Jeep Compass',
       'Mercedes-Benz CLS', 'Datsun redi-GO', 'Toyota Glanza',
       'Porsche Macan', 'BMW X4', 'Maruti Dzire ZXI', 'Volvo XC90',
       'Jaguar F-PACE', 'Audi A8', 'ISUZU MUX', 'Ferrari GTC4Lusso',
       'Mercedes-Benz GLS', 'Nissan X-Trail', 'Jaguar XE', 'Volvo XC60',
       'Porsche Panamera', 'Mahindra Alturas', 'Tata Altroz', 'Lexus NX',
       'Kia Carnival', 'Mercedes-AMG C', 'Lexus RX', 'Rolls-Royce Ghost',
       'Maserati Quattroporte', 'Isuzu MUX', 'Force Gurkha'], dtype=object)
```

In [11]:
```
#EDA
# 1 univariant analysis (means analysis of single variables)

sns.kdeplot(x=df["vehicle_age"],shade=True,color="Red")
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_13672\2581485579.py:4: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(x=df["vehicle_age"],shade=True,color="Red")
```

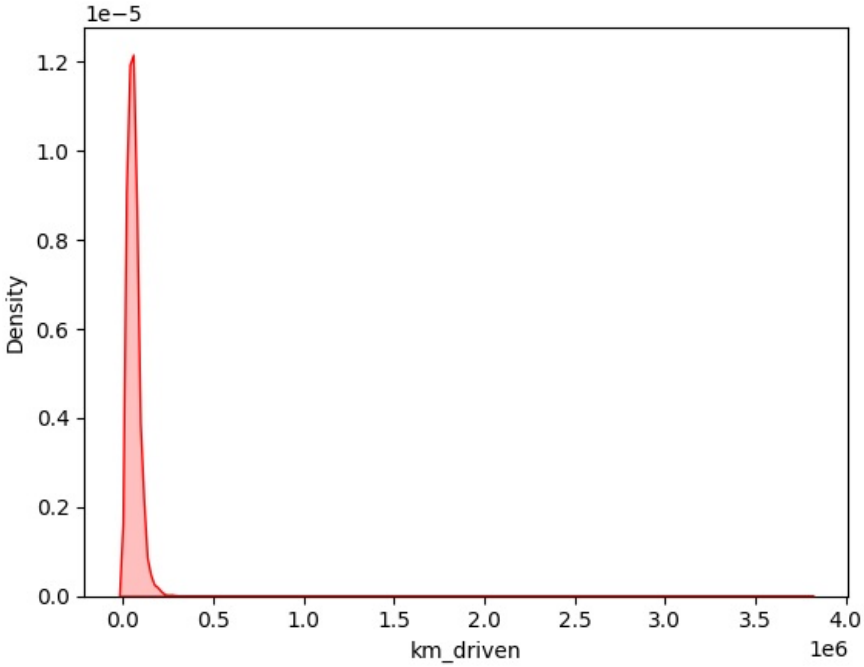
Out[11]:
```
<Axes: xlabel='vehicle_age', ylabel='Density'>
```



In [12]:
```
sns.kdeplot(df["km_driven"],shade= True,color="Red")
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_13672\3422329319.py:1: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(df["km_driven"],shade= True,color="Red")
```

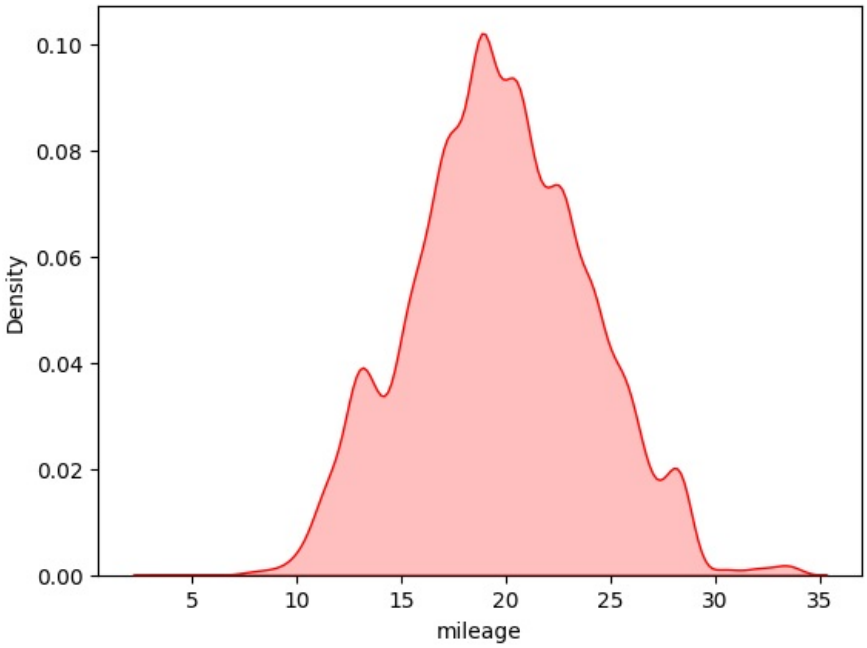
Out[12]:
```
<Axes: xlabel='km_driven', ylabel='Density'>
```



In [13]:
```
sns.kdeplot(df["mileage"],shade= True,color="Red")
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_13672\3639203770.py:1: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(df["mileage"],shade= True,color="Red")
```

Out[13]:
```
<Axes: xlabel='mileage', ylabel='Density'>
```

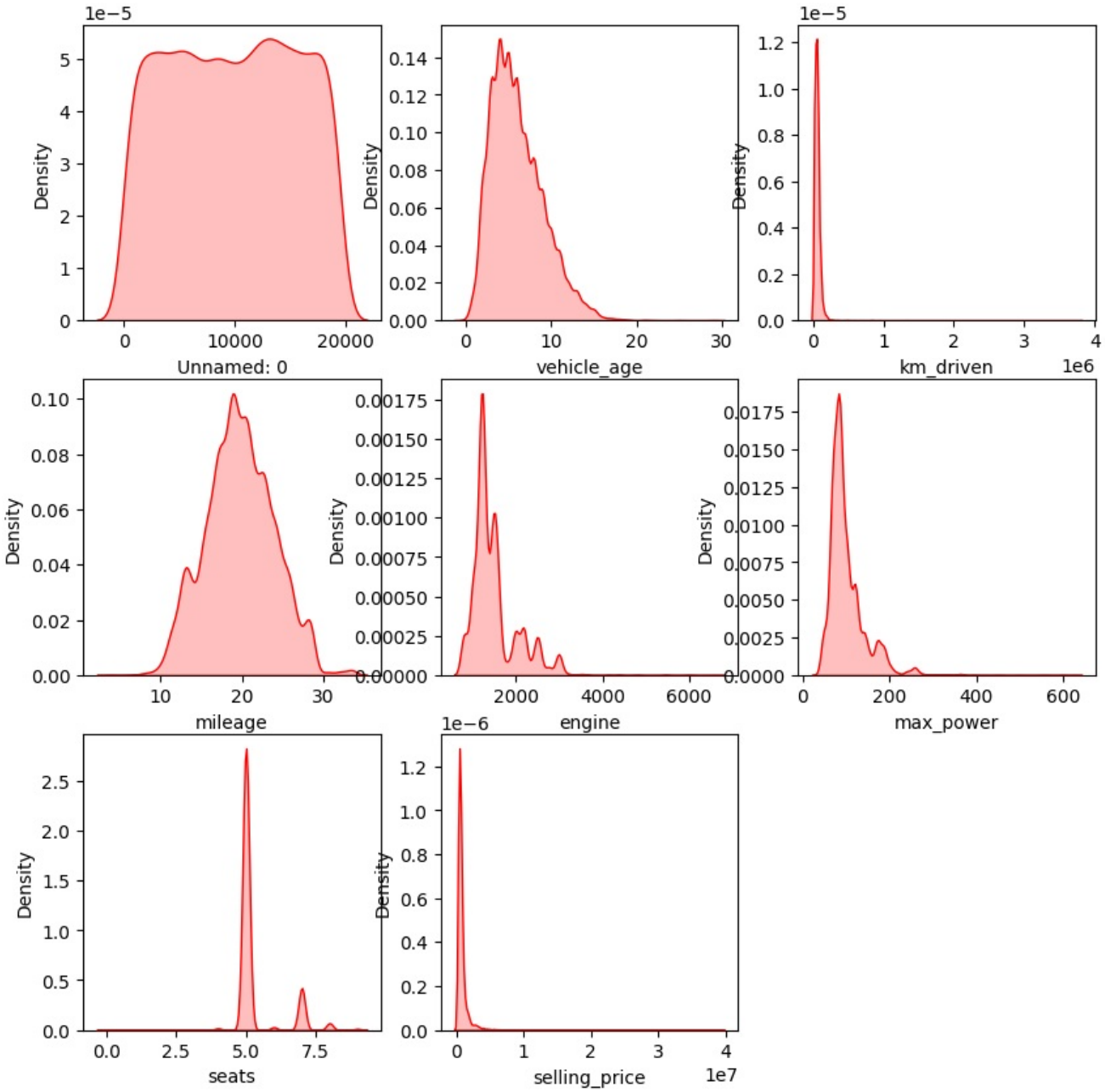


In [14]:
```
#univariate analysis for numerical columns
numrical_Columns = df.select_dtypes(include=np.number).columns.tolist()
numrical_Columns
```

Out[14]:
```
['Unnamed: 0',
 'vehicle_age',
 'km_driven',
 'mileage',
 'engine',
 'max_power',
 'seats',
 'selling_price']
```

In [15]:
```
plt.figure(figsize=(10,10))
for i in range (0,len(numrical_Columns)):
    plt.subplot(3,3,i+1)
    sns.kdeplot(x=df[numrical_Columns[i]],shade=True,color="red")
```

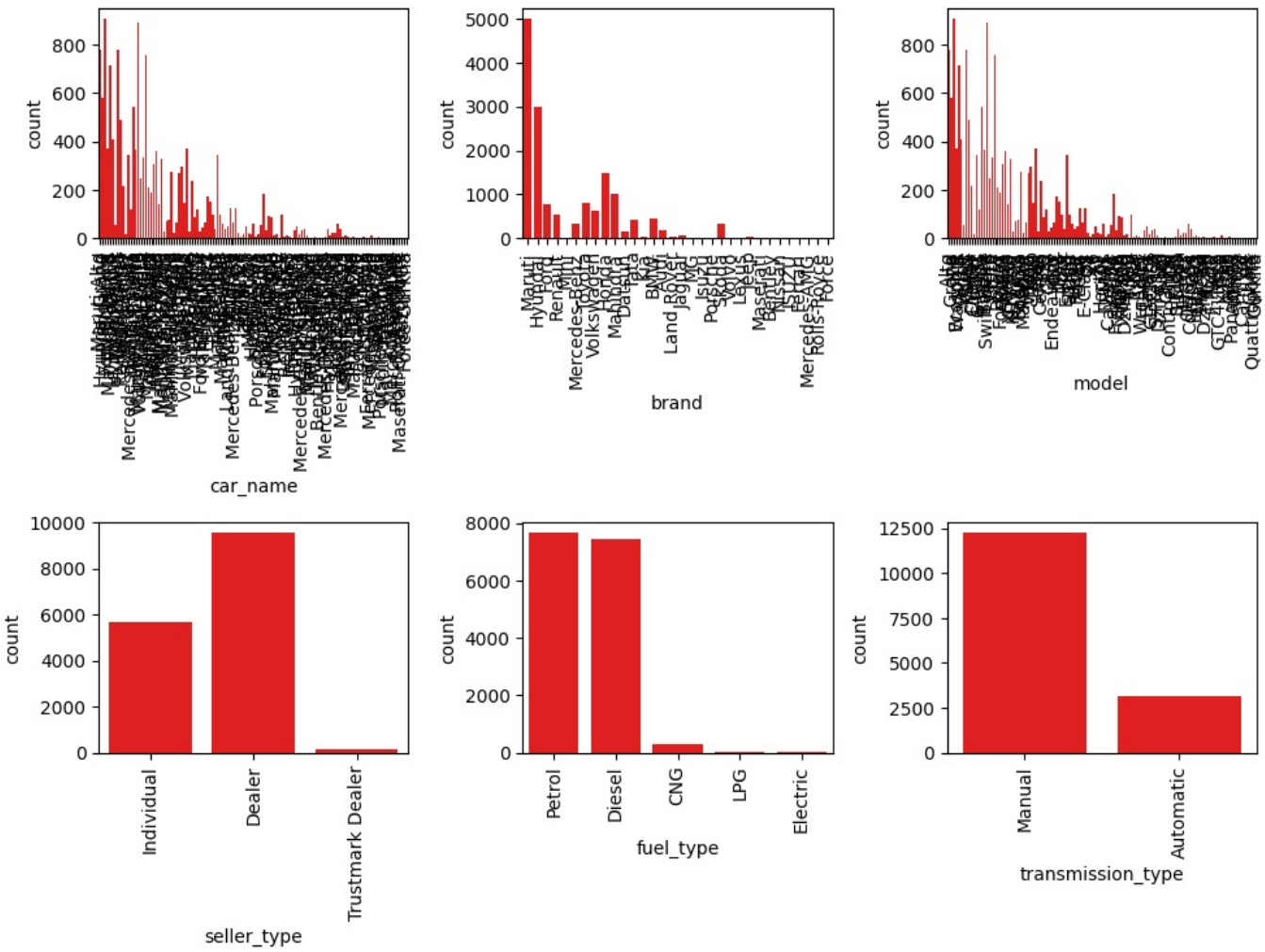
```
C:\Users\DELL\AppData\Local\Temp\ipykernel_13672\680219046.py:4: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(x=df[numrical_Columns[i]],shade=True,color="red")
C:\Users\DELL\AppData\Local\Temp\ipykernel_13672\680219046.py:4: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(x=df[numrical_Columns[i]],shade=True,color="red")
C:\Users\DELL\AppData\Local\Temp\ipykernel_13672\680219046.py:4: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(x=df[numrical_Columns[i]],shade=True,color="red")
C:\Users\DELL\AppData\Local\Temp\ipykernel_13672\680219046.py:4: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(x=df[numrical_Columns[i]],shade=True,color="red")
C:\Users\DELL\AppData\Local\Temp\ipykernel_13672\680219046.py:4: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(x=df[numrical_Columns[i]],shade=True,color="red")
C:\Users\DELL\AppData\Local\Temp\ipykernel_13672\680219046.py:4: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(x=df[numrical_Columns[i]],shade=True,color="red")
C:\Users\DELL\AppData\Local\Temp\ipykernel_13672\680219046.py:4: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(x=df[numrical_Columns[i]],shade=True,color="red")
```



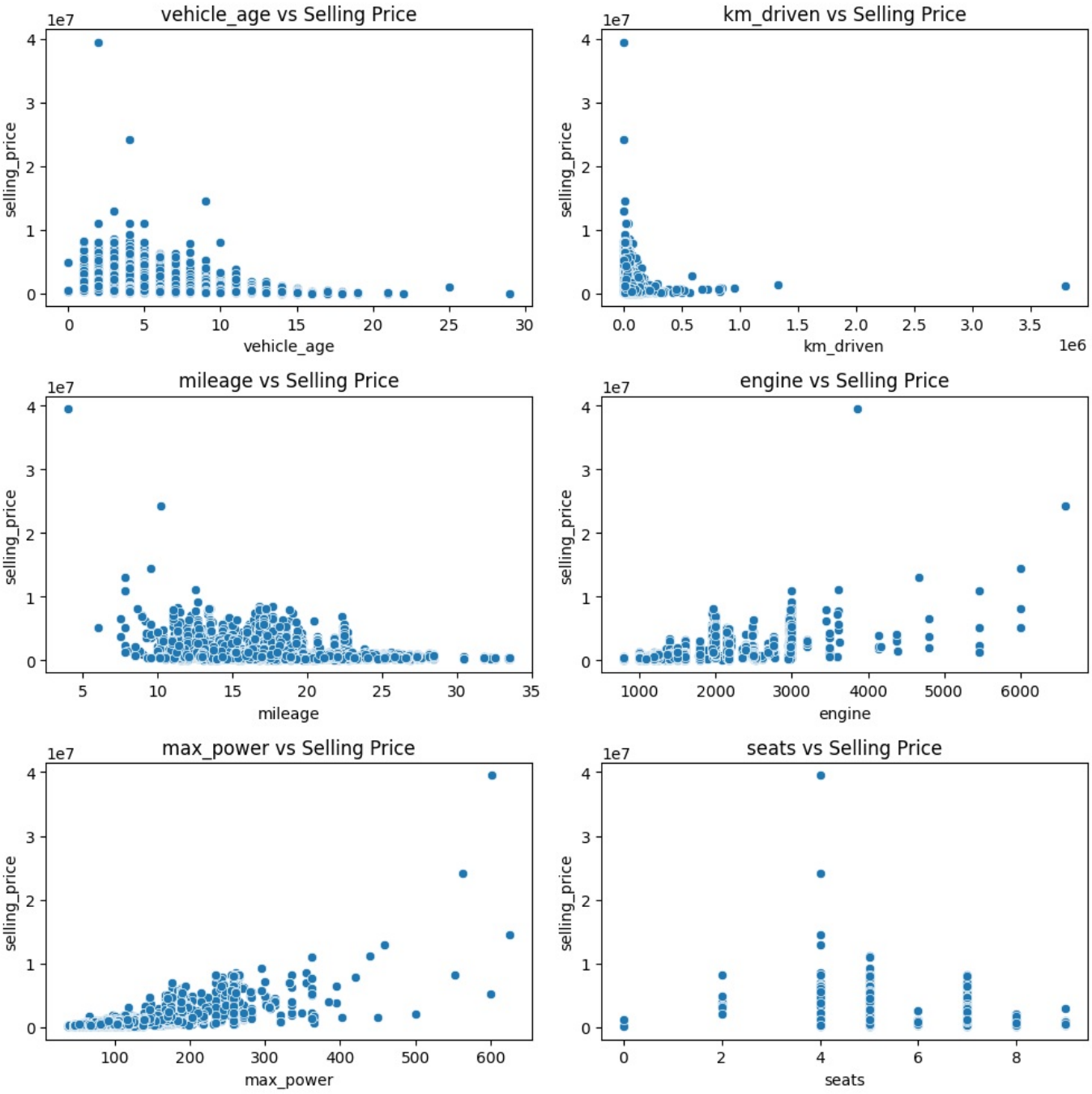
In [16]:
```
#univariate analysis for categorical columns
categorical_columns = df.select_dtypes(include="object").columns.tolist()
print("categorical_columns",categorical_columns)
```

```
categorical_columns ['car_name', 'brand', 'model', 'seller_type', 'fuel_type', 'transmission_type']
```

In [17]:
```
plt.figure(figsize=(10,10))
for i in range (0,len(categorical_columns)):
    plt.subplot(3,3,i+1)
    sns.countplot(x= df[categorical_columns[i]],color = "red")
    plt.xticks(rotation = 90)
plt.tight_layout()
```



In [18]:
```
#bivariant analysis
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10,10))

numerical_iv = ['vehicle_age', 'km_driven', 'mileage', 'engine', 'max_power', 'seats']

for i in range(len(numerical_iv)):
    plt.subplot(3, 2, i+1)
    sns.scatterplot(data=df, x=numerical_iv[i], y='selling_price')
    plt.title(f"{numerical_iv[i]} vs Selling Price")

plt.tight_layout()
plt.show()
```



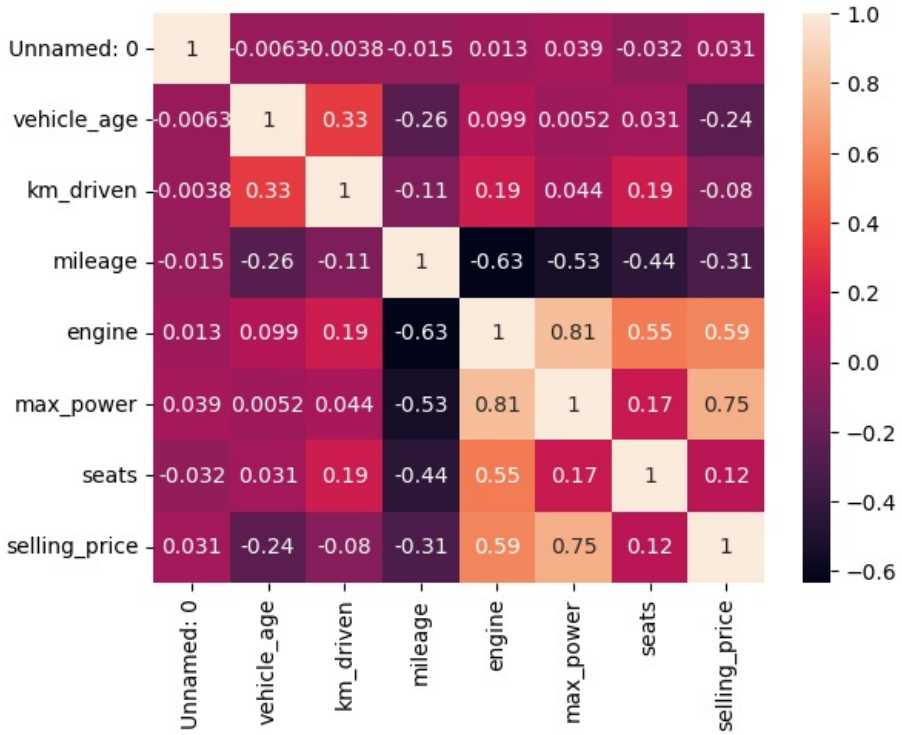
In [19]:
```
#multivariant analysis
#reationship of each variable of all the others
df[numrical_Columns].corr()
```

Out[19]:

	Unnamed: 0	vehicle_age	km_driven	mileage	engine	max_power	seats	selling_price
Unnamed: 0	1.000000	-0.006250	-0.003778	-0.014699	0.012972	0.039367	-0.031832	0.030523
vehicle_age	-0.006250	1.000000	0.333891	-0.257394	0.098965	0.005208	0.030791	-0.241851
km_driven	-0.003778	0.333891	1.000000	-0.105239	0.192885	0.044421	0.192830	-0.080030
mileage	-0.014699	-0.257394	-0.105239	1.000000	-0.632987	-0.533128	-0.440280	-0.305549
engine	0.012972	0.098965	0.192885	-0.632987	1.000000	0.807368	0.551236	0.585844
max_power	0.039367	0.005208	0.044421	-0.533128	0.807368	1.000000	0.172257	0.750236
seats	-0.031832	0.030791	0.192830	-0.440280	0.551236	0.172257	1.000000	0.115033
selling_price	0.030523	-0.241851	-0.080030	-0.305549	0.585844	0.750236	0.115033	1.000000

In [20]:
```
sns.heatmap(data=df[numrical_Columns].corr(),annot=True)
```

Out[20]: <Axes: >



In [21]:
```
model_data  = df.copy()
model_data
```

Out[21]:

	Unnamed: 0	car_name	brand	model	vehicle_age	km_driven	seller_type	fuel_type	transmission_type	mileage	engin
0	0	Maruti Alto	Maruti	Alto	9	120000	Individual	Petrol	Manual	19.70	79
1	1	Hyundai Grand	Hyundai	Grand	5	20000	Individual	Petrol	Manual	18.90	119
2	2	Hyundai i20	Hyundai	i20	11	60000	Individual	Petrol	Manual	17.00	119
3	3	Maruti Alto	Maruti	Alto	9	37000	Individual	Petrol	Manual	20.92	99
4	4	Ford Ecosport	Ford	Ecosport	6	30000	Dealer	Diesel	Manual	22.77	149
...	...	...	...	...	...	...	...	...	...	...	...
15406	19537	Hyundai i10	Hyundai	i10	9	10723	Dealer	Petrol	Manual	19.81	108
15407	19540	Maruti Ertiga	Maruti	Ertiga	2	18000	Dealer	Petrol	Manual	17.50	137
15408	19541	Skoda Rapid	Skoda	Rapid	6	67000	Dealer	Diesel	Manual	21.14	149
15409	19542	Mahindra XUV500	Mahindra	XUV500	5	3800000	Dealer	Diesel	Manual	16.00	217
15410	19543	Honda City	Honda	City	2	13000	Dealer	Petrol	Automatic	18.00	149

15411 rows × 14 columns

In [22]:
```
model_data.drop(labels = ['Unnamed: 0','car_name','brand','model'],axis = 1,inplace = True)
model_data
```

Out[22]:

	vehicle_age	km_driven	seller_type	fuel_type	transmission_type	mileage	engine	max_power	seats	selling_price
0	9	120000	Individual	Petrol	Manual	19.70	796	46.30	5	120000
1	5	20000	Individual	Petrol	Manual	18.90	1197	82.00	5	550000
2	11	60000	Individual	Petrol	Manual	17.00	1197	80.00	5	215000
3	9	37000	Individual	Petrol	Manual	20.92	998	67.10	5	226000
4	6	30000	Dealer	Diesel	Manual	22.77	1498	98.59	5	570000
...	...	...	...	...	...	...	...	...	...	...
15406	9	10723	Dealer	Petrol	Manual	19.81	1086	68.05	5	250000
15407	2	18000	Dealer	Petrol	Manual	17.50	1373	91.10	7	925000
15408	6	67000	Dealer	Diesel	Manual	21.14	1498	103.52	5	425000
15409	5	3800000	Dealer	Diesel	Manual	16.00	2179	140.00	7	1225000
15410	2	13000	Dealer	Petrol	Automatic	18.00	1497	117.60	5	1200000

15411 rows × 10 columns

In [23]:
```
#all categorical data converted into numerical form
model_data = pd.get_dummies(model_data,dtype=int)
model_data
```

Out[23]:

	vehicle_age	km_driven	mileage	engine	max_power	seats	selling_price	seller_type_Dealer	seller_type_Individual	seller_
0	9	120000	19.70	796	46.30	5	120000	0	1	
1	5	20000	18.90	1197	82.00	5	550000	0	1	
2	11	60000	17.00	1197	80.00	5	215000	0	1	
3	9	37000	20.92	998	67.10	5	226000	0	1	
4	6	30000	22.77	1498	98.59	5	570000	1	0	
...	...	...	...	...	...	...	...	...	...	
15406	9	10723	19.81	1086	68.05	5	250000	1	0	
15407	2	18000	17.50	1373	91.10	7	925000	1	0	
15408	6	67000	21.14	1498	103.52	5	425000	1	0	
15409	5	3800000	16.00	2179	140.00	7	1225000	1	0	
15410	2	13000	18.00	1497	117.60	5	1200000	1	0	

15411 rows × 17 columns

In [25]:
```
x = model_data.drop("selling_price",axis=1)
x
```

Out[25]:

	vehicle_age	km_driven	mileage	engine	max_power	seats	seller_type_Dealer	seller_type_Individual	seller_type_Trustmar Deale
0	9	120000	19.70	796	46.30	5	0	1	
1	5	20000	18.90	1197	82.00	5	0	1	
2	11	60000	17.00	1197	80.00	5	0	1	
3	9	37000	20.92	998	67.10	5	0	1	
4	6	30000	22.77	1498	98.59	5	1	0	
...	...	...	...	...	...	...	...	...	
15406	9	10723	19.81	1086	68.05	5	1	0	
15407	2	18000	17.50	1373	91.10	7	1	0	
15408	6	67000	21.14	1498	103.52	5	1	0	
15409	5	3800000	16.00	2179	140.00	7	1	0	
15410	2	13000	18.00	1497	117.60	5	1	0	

15411 rows × 16 columns

In [27]:
```
y=model_data[["selling_price"]]
y
```

Out[27]:

	selling_price
0	120000
1	550000
2	215000
3	226000
4	570000
...	...
15406	250000
15407	925000
15408	425000
15409	1225000
15410	1200000

15411 rows × 1 columns

In [29]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size = 0.2)
```

In [31]:
```
#model building and evaluations

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error,mean_squared_error,r2_score

model = LinearRegression()
regressor = model.fit(x_train,y_train) #trainning the model

pred = regressor.predict(x_test) #testing  the model

print(mean_squared_error(y_true = y_test, y_pred = pred))
print(r2_score(y_test,pred))
```

```
234335796060.66794
0.6623441551523612
```

In [ ]:
```
#R^2 is 0.66 its mean model is not suitable for this data ,for suitable model R^2 value atlear 0.8
```

In [32]:
```
model_data["Predict selling Price"]=regressor.predict(x)
model_data
```

Out[32]:

	vehicle_age	km_driven	mileage	engine	max_power	seats	selling_price	seller_type_Dealer	seller_type_Individual	seller_
0	9	120000	19.70	796	46.30	5	120000	0	1	
1	5	20000	18.90	1197	82.00	5	550000	0	1	
2	11	60000	17.00	1197	80.00	5	215000	0	1	
3	9	37000	20.92	998	67.10	5	226000	0	1	
4	6	30000	22.77	1498	98.59	5	570000	1	0	
...	...	...	...	...	...	...	...	...	...	...
15406	9	10723	19.81	1086	68.05	5	250000	1	0	
15407	2	18000	17.50	1373	91.10	7	925000	1	0	
15408	6	67000	21.14	1498	103.52	5	425000	1	0	
15409	5	3800000	16.00	2179	140.00	7	1225000	1	0	
15410	2	13000	18.00	1497	117.60	5	1200000	1	0	

15411 rows × 18 columns

In [ ]: